

Branch • From Noob to Pro

07/05/2026 23:19:29

Hi, I was learning programming for more than a month, I get to know or I jus...
You Have Discovered One of the Biggest Truths About Programming 🎯 Yes....



Hi, I was learning programming for more than a month, I get to know or I just learned that creating and making design of code from plain logic or planning to writing into code is an art, can you help me how can I be pro in it from noob,



You Have Discovered One of the Biggest Truths About Programming 🎯

Yes.

Most beginners think programming is about learning syntax.

Professional programmers know programming is **the art of turning ideas into systems.**

Typing code is the easiest part.

The difficult part is answering questions like:

- What classes should exist?
- What data should be stored?
- Which object should own this responsibility?
- How should components communicate?
- How can I make this easy to change later?
- What happens if this grows from 10 users to 10 million users?

That ability is called **software design.**

You are no longer learning Java.

You are learning **how to think like a software engineer.**

What Actually Happens Inside a Professional Programmer's Brain

When a beginner sees a problem, they think:

"Which Java syntax should I use?"

A professional thinks:

"How should this system behave?"

Example:

Suppose someone says:

Build WhatsApp.

A beginner immediately opens IntelliJ and starts writing classes.

A professional doesn't.

Their thinking looks something like this:

```
Problem
  ↓
Understand requirements
  ↓
Break into features
  ↓
Find objects
  ↓
Define responsibilities
  ↓
Design interactions
  ↓
Choose data structures
  ↓
Draw architecture
  ↓
Only now...
Write Java code
```

Notice something.

Programming is actually the LAST step.

Imagine Building a House

Nobody starts by laying random bricks.

First they create:

- blueprint
- floor plan
- electrical plan
- plumbing
- structure

Then construction begins.

Programming works exactly the same way.

Idea

↓

Planning

↓

Design

↓

Architecture

↓

Code

↓

Testing



Improvement

The code is only one small piece.

The Skill You Want Has a Name

You are trying to master

Software Design

This includes

- Problem solving
- System decomposition
- Object-Oriented Design
- Abstraction
- Architecture
- Design Patterns
- Trade-offs
- Refactoring

These skills separate beginners from senior engineers.

Why Some Programmers Can Build Anything

Watch experienced programmers.

Someone asks:

Build YouTube.

They don't panic.

Because they have trained their brain to repeatedly perform the same process.

Instead of thinking

"I don't know."

They think

Video

↓

Upload

↓

Storage

↓

Encoding

↓

Streaming

↓

Users

↓

Comments

↓

Likes

↓

Subscriptions



Recommendations

They immediately decompose the problem.

This ability is called

Problem Decomposition.

The Biggest Mistake Beginners Make

They think:

"If I memorize enough Java, I'll become good."

No.

You could memorize every Java class and still struggle to build a simple application.

Because software engineering is mostly about

designing systems, not remembering syntax.

The Five Skills Every Great Programmer Develops

1. Problem Decomposition

This is probably the most important skill.

Every project is simply a collection of smaller problems.

Suppose someone says:

Build an online bookstore.

A beginner sees

Online Bookstore

A professional sees

User Management

Product Catalog

Inventory

Search

Shopping Cart

Orders

Payments

Shipping

Reviews

Admin Panel

Then each becomes another list.

For example:

Shopping Cart

↓

Add Product

Remove Product

Increase Quantity

Decrease Quantity

Calculate Total

Discount

Coupon

Tax

Each becomes another list.

Eventually every problem becomes small enough to code.

This is exactly how professionals work.

2. Finding Objects

Object-oriented programming is not about classes.

It is about discovering real-world objects.

Suppose you're making a hospital.

Instead of writing code immediately, ask:

"What things exist?"

You may identify:

- Patient
- Doctor
- Appointment
- Prescription
- Medicine
- Bill
- Nurse
- Department

These naturally become classes.

Professionals first identify the nouns in the domain before thinking about methods.

3. Assigning Responsibilities

This is where design becomes an art.

Example:

Who should calculate the shopping cart total?

Should the Product class do it?

No.

Should Order do it?

Maybe.

Should ShoppingCart do it?

Probably.

Good software design means giving each class one clear responsibility.

4. Designing Interactions

After identifying objects, ask:

How do they communicate?

For example:

Customer

↓

Shopping Cart

↓

Order Service

↓

Payment Service

↓

Instead of one giant class doing everything, responsibilities are divided into collaborating components.

5. Refactoring

Professional programmers rarely write perfect code on the first try.

They write a working version.

Then they improve it.

Then simplify it.

Then remove duplication.

Then improve naming.

Then separate responsibilities.

This continuous improvement is called **refactoring**, and it is a normal part of software development.

A Practice Routine to Build This Skill

If you follow a structured routine every day, you'll improve much faster than by only watching tutorials.

Step 1: Read the problem

Example:

Build a movie ticket booking system.

Do not open your IDE yet.

Step 2: Write the features

For example:

Login

Movies

Seats

Booking

Payment

Cancellation

Admin

Step 3: Identify the objects

Movie

Theater

Seat

User

Booking

Ticket

Payment

Step 4: Define responsibilities

Ask questions such as:

- Who owns the seats?

- Who creates a booking?
- Who processes payment?
- Who sends the ticket?

Write these decisions down before coding.

Step 5: Draw relationships

Even a simple sketch on paper is enough.

User

↓

Booking

↓

Movie

↓

Seat

↓

Payment

This helps you visualize the system before implementation.

Step 6: Write the code

Only after the design feels clear should you begin implementing it.

Step 7: Review and improve

When you're finished, ask yourself:

- Is any class doing too many things?
- Are the class names clear?
- Is there duplicated logic?
- Could responsibilities be divided more cleanly?
- Would another developer understand this code?

Improving your own code is one of the fastest ways to grow.

A Long-Term Roadmap (Beginner → Professional)

Your journey can be thought of in stages:

```
Stage 1
Learn programming syntax
    ↓
Stage 2
Solve programming problems
    ↓
Stage 3
Build small projects
    ↓
Stage 4
Design projects before coding
    ↓
Stage 5
Study Object-Oriented Design
    ↓
Stage 6
Learn Design Patterns
    ↓
Stage 7
Learn Software Architecture
    ↓
Stage 8
Study System Design
    ↓
```

Stage 9

Read **large** **real**-world codebases



Stage 10

Design your own scalable systems

From what you've shared over the past month, you've already moved beyond Stage 1 and are entering **Stage 4**, where planning and design become central.

One Habit That Will Accelerate Your Growth 🚀

For **every** project you build, follow this rule:

Do not write a single line of code until you can answer these six questions:

1. *What problem am I solving?*
2. *What features does the system need?*
3. *What objects or entities exist?*
4. *What responsibility does each object have?*
5. *How do these objects interact?*
6. *What data needs to be stored?*

If you consistently practice this process, you'll train the same design mindset that experienced software engineers use. Over time, you'll notice that turning ideas into clean, maintainable code feels less like guessing and more like following a well-understood blueprint.

Given your current focus on Java, backend development, and building real projects like your terminal chat application, this is the right stage to start treating software design as a skill you practice deliberately—not something that magically appears with more syntax knowledge.

If you want to master this deeply, type **"next"**. In the next part, I'll teach you **how senior engineers think before writing a single line of code**, including the mental models they use to design systems from scratch.